L2M: An LLM-Driven Lyrics-to-Melody Generation System with Emotion-Aware Alignment

Arafat Hasan
Department of Computer Science and Engineering
Mawlana Bhashani Science and Technology University
Tangail, Bangladesh
opendoor.arafat@gmail.com

Mohammad Motiur Rahman
Department of Computer Science and Engineering
Mawlana Bhashani Science and Technology University
Tangail, Bangladesh
mm73rahman@gmail.com

***Abstract*—We present L2M, a lyrics-to-melody generation system that leverages Large Language Models (LLMs) as the core reasoning engine for cross-modal music composition. Unlike prior approaches that require task-specific training on paired lyrics-melody corpora, L2M adopts a zero-shot prompting strategy: it first applies an LLM to classify the emotional content, tempo, and phrase structure of input lyrics, then uses a second LLM call to generate a syllable-aligned note sequence conditioned on the extracted musical attributes. A deterministic fallback mechanism based on emotion-to-key mappings and melodic contour heuristics ensures 100% operational reliability independent of API availability. The system produces standard music notation outputs (MIDI, MusicXML) and optional audio synthesis (WAV/MP3). Evaluation on a 20-sample benchmark spanning six emotion categories shows 100% syllable-note alignment accuracy, 95% emotion-key consistency, and 100% MIDI validity, with an average end-to-end latency of 3.2 seconds. L2M is released as an open-source Python package with a CLI and programmatic API, providing an accessible baseline for LLM-based music generation research.**

***Index Terms*—Lyrics-to-Melody Generation, Large Language Models, Prompt Engineering, Emotion-Aware Music Generation, Music Information Retrieval, Cross-Modal Generation**

I. Introduction

A. Motivation

Music composition at the intersection of language and music requires simultaneous understanding of semantic meaning, emotional intent, syllabic rhythm, and musical structure. The task of **lyrics-to-melody (L2M) generation**—producing a singable melodic sequence from lyrical text—is particularly demanding, imposing four tightly coupled requirements on any generation system.

Effective lyrics-to-melody generation demands **semantic understanding** of lyrical content and emotional register: the system must interpret not merely the denotative meaning of words but the affective intent and thematic atmosphere embedded in the text. Equally essential is precise **syllabic alignment**—every phonetic syllable must correspond to exactly one musical note, preserving the natural cadence of the sung phrase and ensuring singability.

Beyond these text-level constraints, the output must exhibit **musical coherence**: a well-chosen key, consistent tempo, appropriate mode, and a melodic contour that forms a satisfying phrase arc within the conventions of tonal music. Finally, and most critically, the melody must achieve **emotional consistency**—the affective character of the musical output must reinforce, rather than contradict, the sentiment expressed in the lyrics.

Traditional rule-based approaches lack the flexibility to handle the richness of natural language, while supervised deep learning approaches (Seq2Seq, Transformer-based) require large paired lyrics-melody datasets that are difficult and expensive to curate [1]. The emergence of Large Language Models trained on vast corpora—including music theory texts, song analysis, and transcribed lyrics—opens a compelling alternative: can the emergent reasoning capabilities of LLMs be applied zero-shot to generate musically coherent melodies from lyrics?

B. Problem Statement

This work develops an **end-to-end system** that:

- Accepts arbitrary lyrical text as input with no prior training on lyrics-melody pairs;
- Analyzes emotional content and rhythmic structure using LLM reasoning;
- Generates musically coherent melodies with exact syllabic alignment;
- Exports results in standard music notation formats (MIDI, MusicXML);
- Maintains robust operation via deterministic fallback heuristics when LLM calls fail.

C. Related Work

1) Generative Models for Music Composition: Deep learning approaches to symbolic music generation have employed LSTM, GAN, VAE, and Transformer architectures to model hierarchical temporal structures in music [1]. These foundational methods capture sequential dependencies and latent

musical representations, forming the algorithmic basis for lyric-conditioned systems.

2) Lyrics-to-Melody Generation: **Neural Melody Composition from Lyrics** [2] introduced one of the earliest Seq2Seq neural frameworks for joint lyrics-melody generation using syllable-level alignment on pop music corpora. This work demonstrated feasibility but required large paired datasets.

ReLyMe [3] improved generation quality by explicitly incorporating music theory constraints (tonal relationships, rhythmic patterns) during decoding, addressing a core weakness of purely data-driven models.

Controllable Lyrics-to-Melody Generation [4] introduced style modulation via reference embeddings and memory fusion, enabling user-directed genre and mood control.

SongComposer [5] proposed a fine-tuned LLM that generates lyrics and melodies jointly, trained on large paired song datasets. While achieving high quality, this approach requires extensive domain-specific fine-tuning—a requirement our system eliminates via zero-shot prompting.

Attention-based alignment networks [6] address incomplete or partial lyric inputs by applying cross-attention between text and melody sequences, improving robustness in real-world scenarios.

Contrastive melody-lyrics alignment [7] uses self-supervised contrastive learning to build shared embedding spaces across modalities, demonstrating the importance of strong cross-modal representations.

3) Text-to-Music and Broader Multimodal Generation: Recent comprehensive reviews of AI-enabled text-to-music generation [8] classify systems along symbolic vs. audio synthesis axes and identify persistent challenges: long-term coherence, data scarcity, and limited expressiveness. Systems like MusicLM and AudioCraft demonstrate powerful audio generation from text descriptions, but differ from our task in that they do not enforce syllabic alignment with lyrical text. Transformer-based lyric generation approaches [9] and LSTM-based approaches [10] established baselines for sequential lyric modeling.

4) Research Gap: Existing lyrics-to-melody systems share one or more of the following limitations:

- Require large paired lyrics-melody training corpora for model training;
- Provide no graceful degradation when the model fails at inference time;
- Produce only a single output format (typically MIDI only);
- Apply no explicit emotion-to-key grounding in the generation process;
- Are not available as deployable, open-source software packages.

Our approach directly addresses all five gaps through zero-shot LLM prompting, deterministic fallbacks, multi-format output, explicit emotion-music mappings, and open-source deployment.

II. System Architecture

A. Pipeline Overview

L2M follows a **six-stage sequential pipeline** where each stage produces a validated, typed output consumed by the next (Fig. 1). At each LLM-dependent stage (Stages 2 and 3), a fallback path activates automatically on API failure, timeout, or malformed response, ensuring the pipeline always produces output.

```
Lyrics Input
   |
   v
[Stage 1] Lyrics Parsing
   | Normalized text + phrase structure
   v
[Stage 2] Emotion & Rhythm Analysis <--- LLM Call 1
   | EmotionAnalysis {emotion, tempo, time_sig, phrases[]}
   v
[Stage 3] Melody Structure Generation <--- LLM Call 2
   | MelodyStructure {key, melody[{note, duration, velocity
       }]}
   v
[Stage 4] Internal Representation Building
   | Melody IR {NoteEvent[], tempo, time_sig}
   v
[Stage 5] MIDI / MusicXML Export
   | .mid + .musicxml files
   v
[Stage 6] Audio Rendering (Optional)
     .wav / .mp3 files
```

Fig. 1. L2M Six-Stage Sequential Pipeline.

B. Design Principles

Table I summarizes the five core design principles and their realizations in L2M.

TABLE I
System Design Principles

Principle	Realization
Modularity	Each stage is an independent service with defined I/O types
Type Safety	Pydantic v2 models validate every inter-stage data structure
Robustness	Fallback heuristics at Stages 2 and 3 ensure 100% completion rate
Observability	Structured logging with component-level context at every stage
Extensibility	Service interfaces allow drop-in replacement (LLM provider, audio synthesizer)

C. Data Flow Types

Data transformations across the pipeline are: `str` → `NormalizedLyrics` → `EmotionAnalysis` → `MelodyStructure` → `Melody IR` → `music21.stream.Stream` → MIDI / MusicXML / WAV / MP3 files. Each transition is guarded by Pydantic validation, catching malformed LLM outputs before they propagate downstream.

III. Methodology

A. Stage 1: Lyrics Parsing

The `LyricParser` service performs three preprocessing steps:

- **Normalization**: Collapse whitespace, strip excess punctuation, handle line breaks;
- **Phrase segmentation**: Split on newlines and sentence boundaries into lyric phrases;
- **Syllable estimation**: Vowel-cluster counting with adjustments for silent '*e*' endings, providing an approximate syllable count per phrase.

Syllable estimation serves as a reference for the LLM and for fallback melody generation.

B. Stage 2: Emotion and Rhythm Analysis via LLM

The first LLM call uses a structured few-shot prompt to extract four fields: the dominant emotion (*happy, sad, hopeful, tense, calm, excited, melancholic*); the suggested BPM (validated range 40–200); the time signature (e.g., 4/4, 3/4, 6/8); and per-line syllable breakdowns.

The prompt is structured as: (1) system role assignment as an expert music composer and emotional analyst; (2) JSON output schema specification; (3) three diverse few-shot examples (happy, sad, hopeful); (4) empirically-derived tempo range guidelines per emotion category; (5) target lyrics; and (6) format enforcement instruction.

Fallback (Stage 2): If the LLM call fails or returns invalid JSON, a keyword-matching heuristic detects emotion from a curated lexicon (e.g., "rain", "fade", "dark" $\rightarrow$ *sad*; "dance", "shine", "joy" $\rightarrow$ *happy*), and assigns the median tempo and common time signature for that emotion category.

C. Stage 3: Melody Structure Generation via LLM

The second LLM call generates a syllable-aligned note sequence conditioned on the emotion analysis. The prompt receives the detected emotion, BPM, time signature, total syllable count, and the lyrics.

Key prompt constraints:

- Emotion-to-key mapping guides key selection (positive emotions $\rightarrow$ major keys; negative/tense $\rightarrow$ minor keys);
- **Hard constraint**: "Generate EXACTLY N notes—one per syllable";
- Vocal range: C3–C5 (scientific notation);
- Duration value set: $\{0.25, 0.5, 1.0, 2.0, 4.0\}$ beats;
- Contour guidance: avoid large melodic jumps; create natural phrase arcs.

Chunking for long lyrics: When total syllables exceed a configurable threshold (default: 30), the system splits lyrics into phrase-level chunks and issues sequential LLM calls, carrying the last three notes of each chunk as context for the next to maintain melodic continuity.

Fallback (Stage 3): A deterministic melody generator uses the emotion-to-musical-attribute mapping in Table II and algorithmically constructs a note sequence following the specified contour pattern.

TABLE II
Emotion-to-Musical Attribute Mapping

Emotion	Key	Tempo (BPM)	Contour
Happy	C major	100–120	Ascending
Hopeful	G major	80–100	Wavy (arch)
Sad	A minor	60–80	Descending
Tense	D minor	90–110	Erratic
Calm	F major	60–80	Balanced
Excited	D major	120–140	Ascending (steep)

Contour algorithms are realized as: *Ascending/Descending*—monotonic pitch progression with occasional plateaus based on syllable stress; *Wavy*—alternating up/down movement within a ± 5 semitone range; *Balanced*—Gaussian-weighted note selection centered on the tonic; *Erratic*—uniform random jumps within scale degrees for tension.

D. Stage 4: Internal Representation

`MelodyGenerator.build_melody_ir()` converts the Pydantic `MelodyStructure` into a typed `Melody` dataclass comprising a list of `NoteEvent` objects (scientific pitch notation, duration in quarter-note units, MIDI velocity), tempo, time signature, and key. Note validation enforces pitch range bounds and duration positivity.

E. Stage 5: MIDI/MusicXML Export

The `MIDIWriter` service converts the `Melody` IR to a `music21.stream.Stream` [12], adding tempo markings, key signatures, time signatures, and per-note metadata. The stream is exported to both MIDI (`.mid`) and MusicXML (`.musicxml`) via music21's native exporters.

F. Stage 6: Audio Rendering (Optional)

`AudioRenderer` invokes FluidSynth with a configurable SoundFont file (`.sf2`) to synthesize the MIDI into PCM audio (WAV at 44 100 Hz by default), with optional FFmpeg post-processing for MP3 encoding.

G. Technology Stack

The system is implemented in Python 3.9+. LLM inference uses OpenAI GPT-4o-mini [11] (configurable). Music notation is handled by music21 [12]. Data validation uses Pydantic v2. Audio synthesis uses FluidSynth and FFmpeg. The user interface offers both an argparse CLI and a programmatic Python API. Testing uses pytest with coverage.

IV. Evaluation

A. Evaluation Methodology

Music generation quality is inherently subjective, so we employ both **automatic metrics** (objective, reproducible) and **qualitative analysis** (expert review of sample outputs).

1) Automatic Metrics: Five automatic metrics are computed: (1) *Syllable-Note Alignment Accuracy*—fraction of outputs where note count equals total syllable count; (2) *Emotion-Key Consistency*—fraction using a key consistent with the detected emotion per Table II; (3) *Tempo Appropriateness*—fraction with tempo within the expected BPM range; (4) *MIDI Validity*—fraction of MIDI files successfully parsed by music21; (5) *LLM Success Rate*—fraction of pipeline runs completing without fallback activation.

2) Qualitative Assessment: Three dimensions are evaluated via expert listening: *Melodic Coherence* (smoothness of pitch progression, singability, phrase closure); *Emotional Alignment* (perceived match between lyrical sentiment and musical mood); and *Rhythmic Naturalness* (naturalness of note duration patterns relative to speech rhythm).

B. Test Dataset

We evaluated on 20 lyrical inputs designed to span the emotion space and lyric complexity. The emotion distribution is: Happy (5), Sad (5), Hopeful (4), Tense (3), Calm (2), Excited (1). Lengths span Short (1 line, 5 samples), Medium (2–3 lines, 10 samples), and Long (4+ lines, 5 samples). Style is split equally between Literal/simple and Poetic/metaphorical (10 each).

Table III shows five representative test cases.

TABLE III
SAMPLE TEST CASES

ID	Lyrics	Emotion	Syl.
T1	"The sun will rise again"	Hopeful	6
T2	"Dancing in the moonlight, feeling so alive"	Happy	12
T3	"Memories fade like photographs left in the rain"	Sad	13
T4	"Shadows creeping closer, darkness all around"	Tense	11
T5	"Gentle waves upon the shore, peaceful and serene"	Calm	12

C. Quantitative Results

1) Automatic Metrics: Table IV presents the quantitative evaluation results.

TABLE IV
AUTOMATIC EVALUATION RESULTS ($N = 20$)

Metric	Score	Details
Syllable-Note Alignment	**100%**	20/20 matched exactly
Emotion-Key Consistency	**95%**	19/20; 1 ambiguous case
Tempo Appropriateness	**90%**	18/20; 2 borderline outliers
MIDI Validity	**100%**	20/20 parsed successfully
LLM Success Rate	**85%**	17/20 without fallback

2) Fallback Performance: Of the three cases that activated fallback: two were due to network timeout on LLM API calls, and one was due to a malformed JSON response (notes count mismatch). All three fallback-generated melodies passed MIDI validity checks. Qualitative review rated fallback outputs as "mechanically correct but less expressive" compared to LLM outputs—a known tradeoff of deterministic generation.

TABLE V
MEAN LATENCY PER PIPELINE STAGE

Operation	Mean Time
Emotion analysis (LLM)	$\sim$1.4 s
Melody generation (LLM)	$\sim$1.1 s
MIDI/MusicXML export	$\sim$0.3 s
Audio rendering (WAV)	$\sim$0.4 s
End-to-end (with audio)	**$\sim$3.2 s**

3) Latency Profile: Table V reports mean operation times. The end-to-end pipeline (with audio) averages 3.2 s.

D. Qualitative Analysis of Sample Outputs

T1 — "The sun will rise again" (Hopeful, 6 syllables). Detected key G major, 90 BPM, 4/4. Generated melody: G4(0.5) $\rightarrow$ A4(0.5) $\rightarrow$ B4(1.0) $\rightarrow$ C5(1.0) $\rightarrow$ B4(0.5) $\rightarrow$ A4(1.5). The ascending-arch contour rises to the phrase peak (C5) then resolves. Assessment: musically coherent, emotionally appropriate, all notes diatonic to G major, singable interval range (within a perfect fifth).

T3 — "Memories fade like photographs left in the rain" (Sad, 13 syllables). Detected key A minor, 65 BPM, 4/4. Generated melody begins A4 $\rightarrow$ G4 $\rightarrow$ F4 $\rightarrow$ E4 $\rightarrow$ D4 $\rightarrow$ ...(stepwise descent, resolving on D4). Assessment: melancholic mood captured via stepwise descent and slow tempo; avoids interval leaps inconsistent with the emotional register.

E. Comparison with Related Systems

Table VI compares L2M against prior systems on four criteria.

TABLE VI
COMPARISON WITH RELATED SYSTEMS

System	Training	Output	Fallback	OSS
Bao et al. [2]	Yes	MIDI	No	No
ReLyMe [3]	Yes	MIDI	Partial	No
SongComposer [5]	Yes	MIDI+Lyrics	No	No
L2M (Ours)	**No**	**MIDI,MXL, WAV,MP3**	**Yes**	**Yes**

Advantages of L2M over supervised systems include: zero training data or GPU resources required for deployment; 100% completion rate via hybrid architecture; multiple output formats including playable audio; and sub-5-second end-to-end latency. Current limitations relative to fine-tuned systems include less sophisticated melodic structures (single-voice, no harmony), limited style diversity, and musical creativity bounded by the LLM's zero-shot capability.

V. DISCUSSION

Finding 1 — LLM Viability for Cross-Modal Generation. GPT-4o-mini demonstrates sufficient musical knowledge to generate syllabically aligned, emotionally consistent

melodies in a zero-shot setting, suggesting that LLM pre-training encodes substantial implicit music theory knowledge that can be elicited via structured prompting.

Finding 2 — Prompt Engineering as a Core Technical Contribution. The syllable-count hard constraint ("EXACTLY N notes") was critical: without explicit enforcement, early prompt versions produced note counts deviating by $\pm 30\%$ from the syllable count. Structured JSON schemas and few-shot examples reduced malformed response rates from $\sim 35\%$ (unstructured prompts) to $\sim 15\%$.

Finding 3 — Hybrid Architecture Value. The deterministic fallback (3/20 activations, 15% rate) demonstrates that LLM-only pipelines are insufficient for production systems where reliability is required. The hybrid approach achieves 100% completion with graceful degradation rather than failure.

Finding 4 — Two-Stage Decomposition Benefits. Separating emotion analysis from melody generation provides two benefits: (a) interpretability—users can inspect and override the emotion reading before melody generation; (b) targeted fallback—if only melody generation fails, the correct emotional context is preserved for the fallback generator.

A. Contributions

This work makes the following contributions to the field of LLM-based music generation research:

1) **Two-Stage LLM Pipeline.** A novel decomposition of the lyrics-to-melody task into (a) emotion and rhythm analysis and (b) conditioned melody generation, each handled by separate LLM calls with structured JSON output schemas—enabling interpretability and targeted fallback at each stage.
2) **Emotion-Aware Alignment.** An explicit mapping from detected emotion categories to musical keys, tempo ranges, and melodic contour patterns, grounding LLM-generated output in music theory principles.
3) **Hybrid Reliability Architecture.** Combination of zero-shot LLM generation with deterministic heuristic fallbacks, achieving 100% system reliability across all tested inputs.
4) **Prompt Engineering Methodology.** Structured few-shot prompts with explicit output schemas, constraint enforcement (*"EXACTLY N notes for N syllables"*), and contextual carryover across long-lyric chunks—providing a reusable template for other cross-modal LLM tasks.
5) **Open-Source Baseline.** A production-ready Python package with CLI, programmatic API, multi-format output, and comprehensive logging—providing a reproducible baseline for the research community.

B. Limitations

1) **Harmonic Depth**: Generated melodies are monophonic. Chord progressions, harmonization, and countermelody are outside the current scope.
2) **Cultural Scope**: The emotion-key mapping reflects Western tonal music theory. Non-Western scales, modes, and rhythmic systems are not supported.
3) **Long-Form Coherence**: The chunking strategy maintains local melodic continuity (three-note context carryover) but does not enforce global structural coherence (e.g., return to the tonic at phrase boundaries).
4) **Evaluation Scale**: The 20-sample benchmark enables proof-of-concept validation but is insufficient for statistically robust claims about generalization.
5) **LLM API Dependency**: The primary generation path requires OpenAI API access, incurring per-request costs and network latency.

C. Ethical Considerations

Generated melodies may inadvertently resemble copyrighted works; users should exercise discretion before commercial use. AI-generated content should be clearly disclosed in creative or academic contexts. API costs may create barriers to access in resource-constrained settings; local LLM support would address this.

VI. Conclusion

We presented **L2M**, an LLM-driven lyrics-to-melody system that achieves syllabic alignment, emotional coherence, and operational reliability without any task-specific training. The core technical insight is a two-stage prompting strategy—emotion analysis followed by conditioned melody generation—grounded in an explicit emotion-to-musical-attribute mapping, with a deterministic fallback layer ensuring 100% pipeline completion.

Quantitative evaluation demonstrates 100% syllable-note alignment, 95% emotion-key consistency, and 100% MIDI validity across a 20-sample benchmark. The system is released as an open-source Python package, providing an accessible, reproducible baseline for LLM-based music generation research.

The results suggest that zero-shot LLM prompting, when carefully engineered with structured constraints and few-shot examples, is a viable and practical alternative to supervised training for constrained creative generation tasks—particularly where training data is scarce and reliability is a requirement.

A. Future Work

Future directions include: (1) extending to chord progression generation and multi-voice arrangements; (2) evaluating larger models (GPT-4o, Claude claude-opus-4-6, LLaMA 3) and fine-tuning compact LLMs on paired lyrics-melody data; (3) conducting a large-scale user study ($N \geq 100$) using Mean Opinion Score (MOS) and A/B comparison against ReLyMe and SongComposer; (4) extending the emotion-key mapping to non-Western music systems (maqam, raga, pentatonic) with multi-language syllabification; and (5) developing a web-based GUI with real-time playback and an iterative refinement loop.

ACKNOWLEDGMENT

This work was supported by guidance from Dr. Mohammad Motiur Rahman. The authors acknowledge OpenAI for API access, the music21 development team for music notation infrastructure, and the Pydantic and FluidSynth communities for foundational tools used in this implementation.

REFERENCES

[1] J.-P. Briot, G. Hadjeres, and F. Pachet, *Deep Learning Techniques for Music Generation*. Springer, 2020.

[2] H. Bao et al., "Neural Melody Composition from Lyrics," in *Proc. 26th ACM Int. Conf. Multimedia (MM '18)*, 2018.

[3] Y. Zhang et al., "ReLyMe: Improving Lyric-to-Melody Generation by Incorporating Lyric-Melody Relationships," *arXiv preprint arXiv:2207.05688*, 2022.

[4] Y. Zhang et al., "Controllable Lyrics-to-Melody Generation," *arXiv preprint arXiv:2306.02613*, 2023.

[5] S. Ding et al., "SongComposer: A Large Language Model for Lyric and Melody Generation in Song Composition," *arXiv preprint arXiv:2402.17645*, 2024.

[6] S. Reddy et al., "Deep Attention-Based Alignment Network for Melody Generation from Incomplete Lyrics," *arXiv preprint arXiv:2301.10015*, 2023.

[7] L. Wang et al., "Melody-Lyrics Matching with Contrastive Alignment Loss," *arXiv preprint arXiv:2508.00123*, 2025.

[8] Z. Li et al., "AI-Enabled Text-to-Music Generation: A Comprehensive Review of Methods, Frameworks, and Future Directions," *MDPI Electronics*, vol. 14, no. 6, p. 1197, 2025.

[9] D. Mediakov et al., "Information Technology for Generating Lyrics for Song Extensions Based on Transformers," *Int. J. Modern Education and Computer Science*, vol. 16, no. 1, 2024.

[10] H. Gill et al., "Deep Learning in Musical Lyric Generation," *Yale Undergraduate Research Journal*, vol. 1, no. 1, 2020.

[11] OpenAI, "GPT-4 Technical Report," OpenAI Research, 2024.

[12] M. S. Cuthbert and C. Ariza, "music21: A Toolkit for Computer-Aided Musicology and Symbolic Music Data," in *Proc. Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, 2010.

[13] T. Brown et al., "Language Models Are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.